



HACKTHEBOX

Litter Write-up



Prepared by: sebh24

Sherlock Author(s): Broomey28

Difficulty: **Easy**

Scenario

Khalid has just logged onto a host that he and his team use as a testing host for many different purposes, it's off their corporate network but has access to lots of resources in network. The host is used as a dumping ground for a lot of people at the company but it's very useful, so no one has raised any issues.

Little does Khalid know; the machine has been compromised and company information that should not have been on there has now been stolen – it's up to you to figure out what has happened and what data has been taken.

Artefacts Provided

- suspicious_traffic.pcap - 98c26da32ebc0518f327636f5bad777d

Tools Used:

- Brim

Brim is an open-source desktop application designed to provide an intuitive interface for exploring data in various formats, including network captures (pcaps) and logs. It utilizes the powerful Zed system for analytics and storage, offering efficient search and data transformation capabilities. Brim comes with integrated visualizations, making it easier to analyze network traffic patterns and dig into detailed packet information. It's an excellent tool for security professionals and network administrators who need to navigate large datasets with a combination of structured and unstructured data.

- Wireshark

Wireshark is the most widely known and used network protocol analyzer. It allows you to capture and interactively browse the traffic running on a computer network. It has a rich feature set that includes deep inspection of hundreds of protocols, live capture and offline analysis, and VoIP analysis. Wireshark's powerful filtering and search capabilities make it an invaluable tool for network troubleshooting, software and protocol development, and education in networking and security.

- Tcpdump

Tcpdump is a command-line network analysis tool that captures network packets and displays them in a detailed format. It allows users to capture packets off a network interface and interpret them using a packet decoder. Tcpdump uses packet filtering expressions to target specific types of traffic and can save captured packet data to a file for later analysis. Being lightweight and flexible, tcpdump is a fundamental tool that can be found pre-installed on many Unix-like operating systems and is valuable for anyone looking to perform quick and targeted network capture and analysis.

- Scapy

Scapy is a powerful Python-based interactive packet manipulation program and library. It can forge or decode packets of a wide number of protocols, send them over the wire, capture them, match requests and replies, and much more. Scapy can easily handle tasks like scanning, tracerouting, probing, unit tests, attacks, and network discovery. It provides classes to interactively create packets for a wide range of protocols, and it's a flexible tool that fits well in both research and operational settings.

Initial Analysis

We are provided with a packet capture file, with no other artefacts. Utilising Brim to get a generic overview and run Suricata over the PCAP we only find a variety of unique DNS queries to be of interest. We pop a pin in this to investigate further within Wireshark later in the investigation. The DNS queries are of interest due to the large hex blobs within each sub-domain. This is often an indicator of a tool such as [DnsCat](#).

[illegible]

As we investigate potential data theft, understanding the volume of packets and the protocols generating them is crucial. From the data, we can see common protocols like HTTP and SSH, which are often used for exfiltration; however, there aren't many packets associated with them. On the other hand, DNS accounts for 20% of the entire pcap file, which is unusually high for a 40MB file and serves as our first clue for further investigation. Correlating this to the earlier indicator within the unique DNS queries seen highlights additional need for analysis.

Next, we'll identify which hosts are generating significant amounts DNS traffic:

Ethernet · 18		IPv4 · 244		TCP · 363		UDP · 986					
Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	
192.168.157.144	64377	192.168.157.145	53	9,956	2.153 MiB	481	4,978	994.409 KiB	4,978	1	
192.168.157.144	54211	142.250.180.14	443	3,341	2.222 MiB	531	1,140	485.679 KiB	2,201	1	
192.168.157.144	62672	173.194.129.201	443	6,264	6.829 MiB	641	677	74.072 KiB	5,587	6	
192.168.157.144	56097	172.217.16.238	443	1,112	108.435 KiB	688	552	48.535 KiB	560	5	
192.168.157.144	55860	62.252.115.140	443	3,326	3.608 MiB	706	400	43.722 KiB	2,926	3	
192.168.157.144	62542	62.252.191.206	443	2,906	3.172 MiB	698	326	33.935 KiB	2,580	3	

From the output detailed above, it's evident that there's a substantial amount of DNS traffic between the hosts 192.168.157.144 and 192.168.157.145. We decide to filter the traffic between these two and examine the packets more closely.

At first glance, the interactions appear normal, until we inspect the sub-domains within the Q-Name field.

> Frame 12186: 96 bytes on wire (768 bits), 96 bytes captured (768 bits)

> Ethernet II, Src: VMware_a5:d6:28 (00:0c:29:a5:d6:28), Dst: VMware_6d:01:db (00:0c:29:6d:01:db)

> Internet Protocol Version 4, Src: 192.168.157.144, Dst: 192.168.157.145

> User Datagram Protocol, Src Port: 64452, Dst Port: 53

> Domain Name System (query)

Transaction ID: 0x0d25

> Flags: 0x0100 Standard query

Questions: 1

Answer RRs: 0

Authority RRs: 0

Additional RRs: 0

> Queries

> 617d01661b600a8981.microsofto365.com: type MX, class IN

Name: 617d01661b600a8981.microsofto365.com

[Name Length: 36]

[Label Count: 3]

Type: MX (Mail eXchange) (15)

Class: IN (0x0001)

[Response In: 12187]

The sub-domains are represented in hexadecimal, which is highly unusual and a classic sign of DNS tunnelling. To confirm this, we filtered the packets by query length, multiple subdomains suspicious looking subdomains associated with microsofto365.com.

ip.addr==192.168.157.144 && ip.addr==192.168.157.145 && dns.qry.name.len > 100

No.	Time	Source	src port	Des
-----	------	--------	----------	-----

Using the large query name search, we can see that there are numerous packets with similar suspicious sub domains.

```

Domain Name System (query)
  Transaction ID: 0x0940
  > Flags: 0x0100 Standard query
  Questions: 1
  Answer RRs: 0
  Authority RRs: 0
  Additional RRs: 0
  < Queries
    [truncated]1ae6012ec72fe4b1f14d6963726f736f66742057696e646f7773205b5665.7273696f6e2031302e302e31393034352e323834365d0d0a286329204
    Name: 1ae6012ec72fe4b1f14d6963726f736f66742057696e646f7773205b5665.7273696f6e2031302e302e31393034352e323834365d0d0a286329204d69
    [Name Length: 229]
    [Label Count: 6]
    Type: CNAME (Canonical NAME for an alias) (5)
    Class: IN (0x0001)
    [Response In: 13777]

```

Next we right click and copy the hex within the Name field of Query, placing it in [CyberChef](#) to convert. We see the output as detailed below as numerous comma separated values, which look like some type of PII data. PII, or Personally Identifiable Information, refers to any data that can be used to identify a specific individual either on its own or when combined with other relevant data. This type of information is crucial because it can include direct identifiers that connect data to a particular person without needing additional information.

```

Output
&gsoH FS îÈpw«
Lake Morganhaven, NV 15843",keith44@gmail.com,1984-01-05 cr
643,Education administrator,"Evans, FF SI 6ENQ FF

```

At this stage we are able to confirm that within the PCAP there is likely the exfiltration of PII data over a DNS tunnel between the previously identified hosts. Moving forward we must put together a method of querying the data and attempting to output it to confirm which data has been stolen. For this usecase we will utilise Scapy, however prior to doing so lets reduce the size of our pcap by parsing out all the DNS traffic.

```

sebh24@rabat:~/ $ tcpdump -nr suspicious_traffic.pcap port 53 -w dns.pcap
reading from file suspicious_traffic.pcap, link-type EN10MB (Ethernet), snapshot
length 262144

```

Next we create the Python3 script to utilise Scapy as detailed below:

```

from scapy.all import *
def is_hex(s):
    """Check if the string consists only of hexadecimal characters."""
    try:
        int(s, 16)
        return True
    except ValueError:
        return False
r = rdpcap('dns.pcap')
with open("output.txt", "w") as myfile:
    c = b""

```

```

for packet in r:
    if packet.haslayer(DNSQR):
        a = packet[DNSQR].qname
        no9 = a[18:]
        b = no9.replace(b'microsoft365.com.', b'')
        if not b or b == c:
            continue
        c = b
        try:
            hex_chars = ''.join(ch for ch in b.decode(errors='ignore') if ch
in '0123456789abcdefABCDEF')
            ascii_str = bytes.fromhex(hex_chars).decode('utf-8',
errors='ignore')
            myfile.write(ascii_str + '\n')
        except ValueError as e:
            print(f"Error converting hex to ASCII: {e}")

```

Okay now lets breakdown the script and its features:

- `from scapy.all import *`: Imports all modules from Scapy, a powerful packet manipulation library.
- `def is_hex(s):`: Defines a function `is_hex` that checks if a given string `s` contains only hexadecimal characters.
 - `try:`: Begins a try block to attempt the conversion of `s` to an integer with a hexadecimal base.
 - `int(s, 16)`: Tries to convert the string `s` to an integer with base 16 (hexadecimal).
 - `return True`: Returns `True` if the conversion succeeds, meaning `s` is a valid hex string.
 - `except ValueError:`: Catches a `ValueError` that occurs if `s` is not a valid hexadecimal string.
 - `return False`: Returns `False` if a `ValueError` is caught, indicating `s` is not a valid hex string.
- `r = rdpcap('dns.pcap')`: Reads a pcap file named 'dns.pcap' into the variable `r` using Scapy's `rdpcap` function.
- `with open("output.txt", "w") as myfile:`: Opens a file named 'output.txt' in write mode and assigns it to the variable `myfile`. The file is guaranteed to close when the block ends.
- `c = b""`: Initialises a variable `c` as an empty byte string to store the previous DNS query for comparison.
- `for packet in r:`: Loops through each packet in the pcap file.
 - `if packet.haslayer(DNSQR):`: Checks if the current packet has a DNS Question Record (DNSQR) layer.
 - `a = packet[DNSQR].qname`: Retrieves the queried name (`qname`) from the DNSQR layer and stores it in variable `a`.
 - `no9 = a[18:]`: Slices the `qname` starting from the 19th character to remove the first 18 characters, which are assumed to be non-hexadecimal, storing it in `no9`.

- `b = no9.replace(b'microsoft365.com.', b'')`: Replaces the domain 'microsoft365.com.' with an empty byte string in `no9`, assuming this is a common suffix to be removed, and stores it in `b`.
- `if not b or b == c:`: Checks if `b` is either empty or identical to `c`, the previous DNS query.
- `continue`: Skips the rest of the loop iteration if `b` is empty or the same as the previous query.
- `c = b`: Sets `c` to the current DNS query for comparison in the next iteration.
- `try:`: Begins another try block to handle potential errors in hex conversion.
 - `hex_chars = ''.join(ch for ch in b.decode(errors='ignore') if ch in '0123456789abcdefABCDEF')`: Decodes `b` and filters out any characters that are not valid hexadecimal digits, joining them into a string `hex_chars`.
 - `ascii_str = bytes.fromhex(hex_chars).decode('utf-8', errors='ignore')`: Converts `hex_chars` from a hex string to bytes, then decodes it into an ASCII string, storing it in `ascii_str`.
 - `myfile.write(ascii_str + '\n')`: Writes the `ascii_str` followed by a newline to the output file.
- `except ValueError as e:`: Catches any `ValueError` that occurs during hex to ASCII conversion.
 - `print(f"Error converting hex to ASCII: {e}")`: Prints an error message along with the exception details if an error occurs.

We then run the script utilising Python3:

```
sebh24@rabat:~/ $ python3 test.py
Error converting hex to ASCII: non-hexadecimal number found in fromhex() arg at position 3
Error converting hex to ASCII: non-hexadecimal number found in fromhex() arg at position 1
Error converting hex to ASCII: non-hexadecimal number found in fromhex() arg at position 1
```

Upon opening the output.txt file within our text editor of choice, we are able to view the commands ran clearly.

```
command (DESKTOP-UMNCBE7)
```

```
cmd.exe (DESKTOP-UMNCBE7)
```

```
Microsoft Windows [Version 10.0.19045.2846]  
(c) Microsoft Corporation. All rights reserved.
```

```
Microsoft Windows [Version 10.0.19045.2846]  
(c) Microsoft Corporation. All rights reserved.
```

```
C:\Users\test\Downloads>
```

```
whoami  
desktop-umncbe7\test
```

```
C:\Users\test\Downloads>
```

```
dir  
Volume in drive C has no label.  
Volume Serial Number is 503A-D127  
  
Directory of C:\Users\test\Downloads  
  
30/04/2023  11:28    <DIR>          .  
30/04/2023  11:28    <DIR>          ..  
06/05/2021  14:16           1,447,178 7z1900-x64 (1).exe  
01/03/2021  17:29           1,447,1
```

Scrolling down the outputted file, we are able to locate the beginning of the data exfiltration on line 911.

```

910
911 type "C:\Users\test\Documents\client data optimisation\user details.csv"
912 ,job,company,ssn,resid
913 ence,current_location,blood_group,website,username,name,sex,address,mail,birthdate
914 0,Chief Tec
915 hnology Officer,Bennett Group,725-79-1073,"5021 Mary Glens
916 Timothyville, TN 14945","(Decimal('5
917 7.8730225'), Decimal('51.378019'))",A-,['https://www.lamb.org/'],michellelynch,David Davis,M,"5
918
919 82 Hernandez Heights Apt. 177
920 Hillhaven, SC 87188",belljennifer@hotmail.com,1980-02-21
921 1,Learn
922 ing mentor,"Solis, Burns and Ward",384-57-7026,"097 Caleb Garden Suite 180
923 Stevenfort, OR 74764
924 ","(Decimal('-24.0063365'), Decimal('112.397315'))",A+,['https://www.gutierrez.net/'],sowen,Ver
925 onica Smith,F,"12772 Thompson Mountains
926 Lake Rhondafurt, VT 79134",ericchase@yahoo.com,1946-09-
927 19
928 2,Quarry manager,Smith Inc,848-03-8333,"86789 Harper Mission Suite 354
929 North Destiny, MA 95
930 710","(Decimal('-3.582688'), Decimal('29.107017'))",AB+,['https://smith.com/'],teresagriffith,J
931 ames Ayala,M,"2485 Crawford Plains
932 Scottmouth, WI 90021",shane Kelley@hotmail.com,1957-06-14
933 3,
934 Quarry manager,"Vasquez, Benton and Shaw",376-18-6853,"50229 Waller Junctions Apt. 911
935 Lake Eri
936 c, WY 43145","(Decimal('-76.5533665'), Decimal('10.902613'))",AB-,['https://waller.com/', 'htt
937 p://silva.biz/', 'http://www.jones.biz/', 'https://morgan.com/'],jenniferrogers,Anna Salazar,F
938
939 ,"681 Kelly Track
940 Martinview, VA 84793",kristagonzales@gmail.com,1934-12-01
941 4,Restaurant manag
942 er,Manning PLC,752-07-3061,"96779 Henry Skyway Apt. 390
943 Goodwinview, ME 91839","(Decimal('86.66
944 6571'), Decimal('-117.531750'))",A+,['https://lewis.net/'],javier23,Courtney Norman,F,"Unit 284
945 2 Box 5492
946 DPO AP 96739",thomas41@yahoo.com,1930-09-25
947 5,Furniture conservator/restorer,Warren
948 -Frost,096-17-8420,"6989 Christopher Trail Suite 529
949 North Cynthia, OR 16415","(Decimal('-9.313

```

As detailed above we confirm the TA reading the "C:\Users\test\Documents\client data optimisation\user details.csv" file, followed by the retrieval of the contents of the file via the unencrypted channel. Prior to confirming the number of user's to have had their data retrieved, lets provide a brief summary on the attacker activity within the session:

1. The attacker initially executes whoami.

```

74 C:\Users\test\Downloads>
75
76 whoami
77 desktop-umncbe7\test
78

```

2. The attacker lists the contents of the Downloads directory.

```

220 C:\Users\test\Downloads>
221 dir
222 Volume in drive C has no label.
223 Volume Serial Number is 503A-D127

```

3. The attacker renames the dnscat binary to win_installer.exe.

```

ren dnscat2-v0.07-client-win32.exe win_installer.exe

C:\Users\test\Downloads>

dir

```

4. The attacker traverses to the users home directory & lists again.

```

8   cd ..
9
0   C:\Users\test>dir
1   Volume in drive C has no label.
2   Volume Serial Number is 503A-D127
3

```

5. The attacker navigates into the OneDrive directory.

```

0   cd OneDrive
1
2   C:\Users\test\OneDrive>
3   dir
4   Volume in drive C has no label.
5   Volume Serial Number is 503A-D127
6

```

6. The attacker navigates into the Documents directory.

```

09  cd documents
10
11  C:\Users\test\Documents>
12  dir
13  Volume in drive C has no label.
14  Volume Serial Number is 503A-D127
15

```

7. The attacker navigates into the `client data optimisation` directory.

```

cd "C:\Users\test\Documents\client data optimisation\"

C:\Users\test\Documents\client data op
timisation>

dir
Volume in drive C has no label.
Volume Serial Number is 503A-D127

Directory of C:\Us
ers\test\Documents\client data optimisation

30/04/2023  11:09    <DIR>          .
30/04/2023

3  11:09    <DIR>          ..
30/04/2023  11:09                    0 Info on the ingest process.t
xt
30/04/2023  11:09                    7 README.rtf
30/04/2023  10:58            239,714 user de
tails.csv
| | | | | | | | 3 File(s)          239,721 bytes
| | | | | | | | 2 Dir(s)  24,499,978,2
40 bytes free

```

8. The attacker reads the contents of the `user details.csv`.

```

type "C:\Users\test\Documents\client data optimisation\user details.csv"
,job,company,ssn,residence,current_location,blood_group,website,username,name,sex,address,mail,birthdate
0,Chief Tec
hnology Officer,Bennett Group,725-79-1073,"5021 Mary Glens
Timothyville, TN 14945","(Decimal('5
7.8730225'), Decimal('51.378019'))",A-,['https://www.lamb.org/'],michellelynch,David Davis,M,"5
82 Hernandez Heights Apt. 177
Hillhaven, SC 87188",belljennifer@hotmail.com,1980-02-21
1,Learn
ing mentor,"Solis, Burns and Ward",384-57-7026,"097 Caleb Garden Suite 180
Stevenfort, OR 74764
","(Decimal('-24.0063365'), Decimal('112.397315'))",A+,['https://www.gutierrez.net/'],sowen,Ver
onica Smith,F,"12772 Thompson Mountains
Lake Rhondafurt, VT 79134",ericchase@yahoo.com,1946-09-
19

```

Finally, we move into the stage of our analysis where we assess the impact to our userbase. The format of our csv file is as follows:

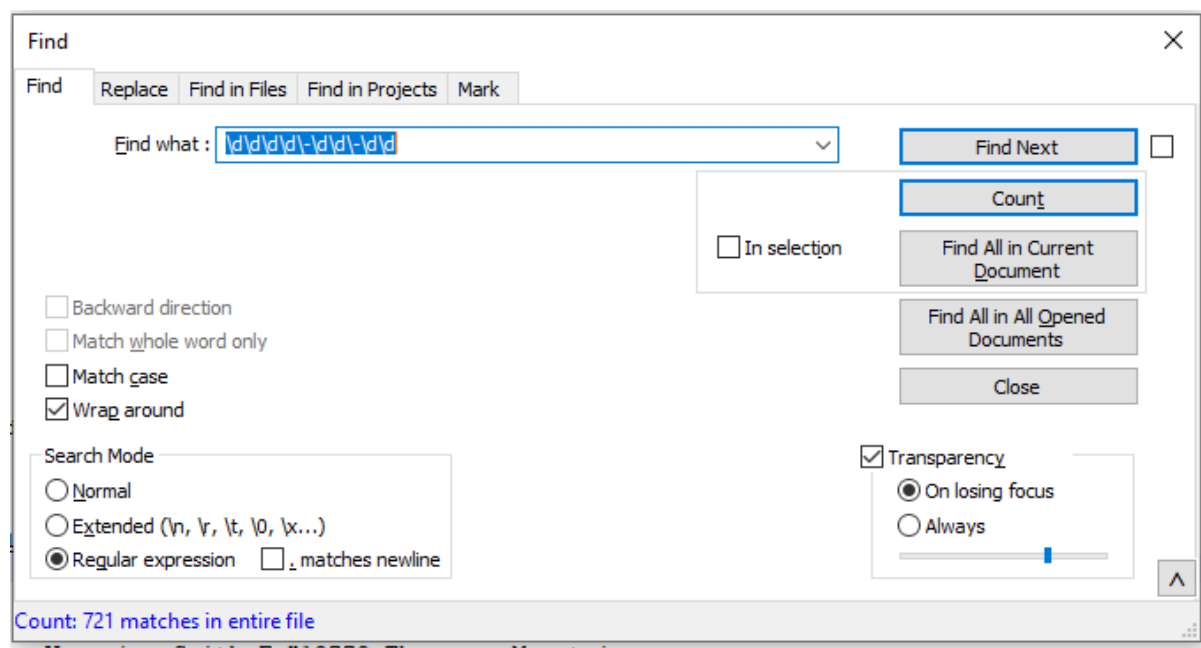
```

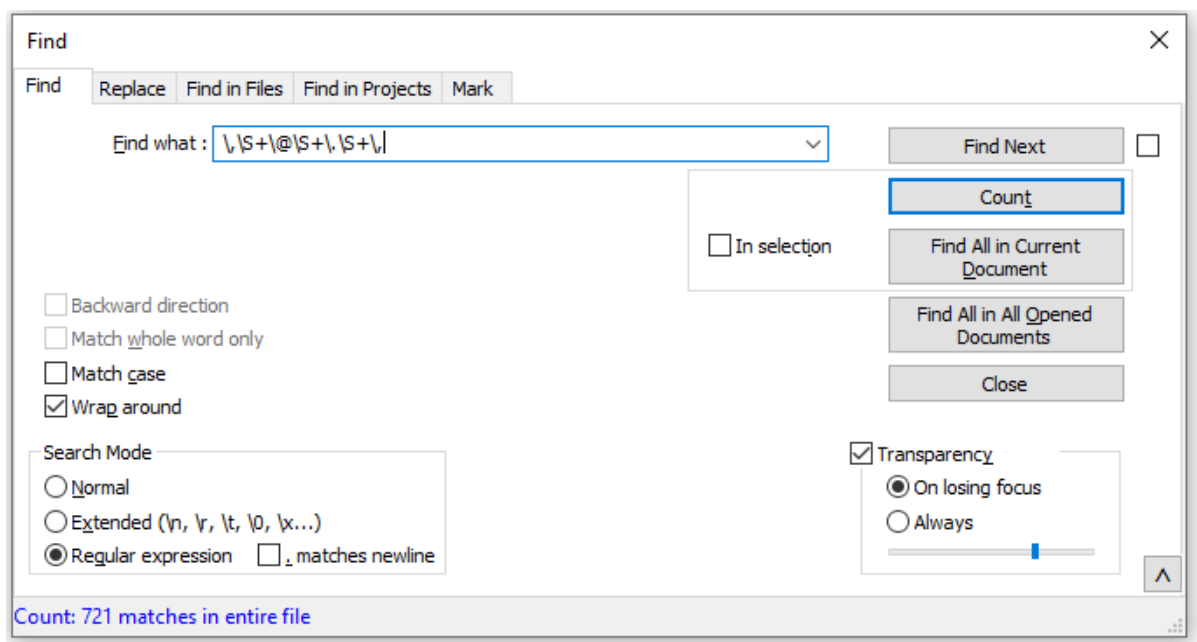
job,company,ssn,residence,current_location,blood_group,website,username,name,sex,address,mail,birthdate

```

Moving forward we will confirm the number of users details stolen by corroborating two likely unique field values - the date &

We open up Notepad++ and use the Find functionality to confirm and corroborate the numbers.





At this stage we have confirmed 721 user's have had their data ex-filtrated by the attacker.

Questions

1. At a glance, what protocol seems to be suspect in this attack?

Answer: DNS

2. There seems to be a lot of traffic between our host and another, what is the IP address of the suspect host?

Answer: 192.168.157.145

3. What is the first command the attacker sends to the client?

Answer: whoami

4. What is the version of the DNS tunneling tool the attacker is using?

Answer: 0.07

5. The attackers attempts to rename the tool they accidentally left on the clients host. What do they name it to?

Answer: win_installer.exe

6. The attacker attempts to enumerate the users cloud storage. How many files do they locate in their cloud storage directory?

Answer: No

7. What is the full location of the PII file that was stolen? (Format: "C:\path\to\file\file.ext")

Answer: C:\users\test\documents\client data optimisation\ user details.csv

8. Exactly how many customer PII records were stolen? (format: integer e.g "65")

Answer: 721